IN 213 **Énoncé**

Sémantique dénotationnelle

Introduction

Le but de ce TP est de programmer la partie sémantique d'un interprète du langage PCF. Les principaux composants du programme vous sont donnés : analyseurs lexical et syntaxique, boucle principale, ainsi que quelques éléments de la partie sémantique.

Les fichiers desquels vous devez partir sont dans dans le répertoire `TDs/05` de la page du cours et sont les suivants : Les fichiers sont les suivants :

- `pcflex.mll` : l'analyseur lexical
- `pcfast.ml` : le type des arbres de syntaxe abstraite, et leur imprimeur
- `pcfparse.mly` : l'analyseur syntaxique
- `pcfsem.ml` : les éléments de la sémantique dénotationnelle du langage, c'est-à-dire
  - le type `pcfval` des valeurs sémantiques
  - des indications relatives aux différents eléments à produire (environnements, fonction sémantique).
- `pcfloop.ml` : la boucle principale,
- `Makefile` : indispensable !

Compiler et recompiler

La commande `make` permet de reconstruire l'exécutable `pcfloop`, après chacune de vos modifications.

Note : La toute première fois, après récupération des fichiers, vous devrez créer un fichier (vide) `.depend` dans le répertoire où vous aurez mis les fichiers. Cela permettra une gestion automatisée des dépendances de compilation (quoi recompiler quand un fichier a été modifié). Vous invoquerez alors `make depend` afin que ces dépendances soient calculées initialement. Lorsque vous modifierez vos fichiers, si vous utilisez une fonction d'une unité de compilation non encore utilisée dans un fichier, la compilation pourra échouer car les dépendances entre fichiers ne seront peut-être pas à jour (en fait, c'est très peu probable car le squelette impose déjà les dépendances nécessaires). Dans ce cas, re-invoquez `make depend`.

1 Travail à réaliser

1.1 Traitement des constantes et des opérateurs

Q1 Complétez `pcfsem.ml` de sorte à ce que le traitement des constantes entières, booléennes, de la conditionnelle, et des opérateurs unaires et binaires soit effectif.

La fonction sémantique devra garder son paramètre `rho` même s'il n'est jamais utilisé à ce stade (il le sera à partir de la question suivante).

À chaque ajout, recompilez et testez.

1.2 Traitement des identificateurs

On vous propose dans ce TP de traiter les environnements comme des fonctions partielles des identificateurs vers les valeurs sémantiques. L'environnement initial ne connaît aucun nom.

Q2 Écrivez la fonction `extend`, telle que `extend rho x v` représente l'environnement qui associe `v` à `x` et `env (y)` à tout `y` différent de `x`.

Q3 Mettez `extend` à l'œuvre en ajoutant le traitement des identificateurs et de la construction `let ... in ....`

1.3 Fonctions et applications

Les valeurs sémantiques des fonctions de PCF sont essentiellement des fonctions OCaml des valeurs sémantiques vers les valeurs sémantiques. C'est le sens du cas

| *Funval* **of** (*pcfval* −> *pcfval*)

de la définition du type `pcfval`.

Q4 Complétez le traitement des fonctions et des applications.

1.4 Déclarations récursives

La syntaxe des définitions récursives permet de définir des *fonctions* récursives locales. Vous avez vu, durant le cours, que la sémantique d'une fonction récursive utilisait un *environnement récursivement défini*.

Q5 En utilisant le fait que les environnements sont représentés par des fonctions (qui peuvent donc être définies récursivement en OCaml), complétez votre fonction sémantique en donnant la sémantique de ces fonctions récursives locales.

1.5 S'il vous reste du temps ...

... enrichissez votre langage à votre gré : traitement des chaînes de caractères, environnement primitif plus riche, définitions globales, etc.